

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_34138e24-10c\agent-a00ffe93278302ee9.jsonl`
- SHA-256: `59eb968953ce961db78cb2a6cf0a2e9053dc6d8103bafa86cdfa9bf28954d37c`
- Source modified: `2026-06-18T01:17:51+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_34138e24-10c`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T01:10:52.224Z)

You are reviewing a small bug-fix commit in a Python repo (badminton-highlight-indexer). Work in this git worktree (read-only ? DO NOT modify, commit, or push anything):

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix
First run: `git -C "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix" diff origin/master..HEAD` to see the exact change. Then read the surrounding code in backend/eval/ablation.py (load_videos, _is_fusion_schema, litmus_config, default_signals) and tests/test_ablation.py (the litmus tests, _golden_present, the new regression tests, _write_learned_corpus).
```

CONTEXT: The fix makes ablation.load_videos() skip "windowing-only" golden_features.json files (leaner schema: start/end + velocity stats, NO shuttle/person/audio cue dicts) so a MIXED corpus (6 full-fusion files + 5 leaner files) loads only the 6 ablatabile files instead of crashing with KeyError: 'shuttle'. The litmus must still reproduce the recorded Gate-A numbers (?F1?0.074, 6/6 wins, p?0.031) on exactly those 6 files.

Be adversarial: actively try to find a reason this should NOT be merged as-is. Report concrete, verifiable findings only (cite file:line). Do not invent issues. For each finding give a severity: must-fix (a real bug / regression / breaks the litmus), should-fix, nit, or none. Set blocks_merge=true ONLY if you found a genuine must-fix. Return the structured object.

YOUR LENS ? edge cases & robustness:

- Empty candidate list; a file where SOME candidates have shuttle and others don't (mixed within one file); a file with shuttle but missing person; malformed/garbage JSON.
- _golden_present()'s try/except (OSError, ValueError) ? does it swallow the right errors and not mask real ones? Is json.load's error a ValueError subclass?
- The >=6 gate semantics: what happens if the fusion-file count drifts above/below 6 ? is that a latent fragility introduced or pre-existing, and does this change make it better or worse?
- Does skipping files silently (only an info log) risk hiding a real problem from a user running the CLI on a dir they expect to be all-fusion?

2. assistant (2026-06-18T01:10:55.258Z)

I'll start by examining the exact change in this commit, then review the surrounding code.

3. user (2026-06-18T01:10:56.857Z)

```
diff --git a/backend/eval/ablation.py b/backend/eval/ablation.py
index fec3e7d..968eefd 100644
--- a/backend/eval/ablation.py
+++ b/backend/eval/ablation.py
@@ -383,16 +383,37 @@ def litmus_config() -> AblationConfig:
# ----- corpus I/O
```

```

+def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
+    """True iff every candidate row carries the per-cue feature dicts the ablation scorer reads
+    (`shuttle` + `person`; `audio` stays optional ? older JSONs predate it).
+
+    The velocity-windowing / distill path persists a LEANER ``*_golden_features.json`` row
+    (`start`/`end` + velocity stats, no cue dicts) for clips that never went through fusion
+    feature extraction. `load_videos` SKIPS those rather than crash on `c["shuttle"]` ? so
+    a
+    mixed corpus (some fusion files, some windowing-only) loads the ablatable subset cleanly.
+    An empty candidate list is not ablatable either."""
+    return bool(candidates) and all("shuttle" in c and "person" in c for c in candidates)
+
+
+def load_videos(features_dir: str) -> List[VideoCandidates]:
+    """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals + shuttle +
+    person + audio columns + golden gts). Audio columns are optional (older JSONs predate the
+    audio augment) ? a missing block defaults those columns to 0.0 so the loader never crashes,
+    mirroring ``experiment._feature_column``'s tolerance."""
+    """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a ``VideoCandidates``
+    (intervals + shuttle + person + audio columns + golden gts). Audio columns are optional
+    (older JSONs predate the audio augment) ? a missing block defaults those columns to 0.0 so
+    the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.
+
+    Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the schema the
+    R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus loads the
+    ablatable subset cleanly (see ``_is_fusion_schema``)."""
+    videos: List[VideoCandidates] = []
+    for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
+        with open(path, encoding="utf-8") as f:
+            d = json.load(f)
+            cands = d["candidates"]
+        if not _is_fusion_schema(cands):
+            logger.info("ablation.load_videos: skipping %s ? windowing-only schema "
+                        "(no shuttle/person cue dicts), not an ablatable fusion-feature file",
+                        os.path.basename(path))
+            continue
+        intervals = [(float(c["start"]), float(c["end"])) for c in cands]
+        features: Dict[str, List[float]] = {}
+        for fn in SHUTTLE_COLUMNS:
diff --git a/tests/test_ablation.py b/tests/test_ablation.py
index b29ecbb..224d497 100644
--- a/tests/test_ablation.py
+++ b/tests/test_ablation.py
@@ -357,6 +357,48 @@ def test_load_videos_tolerates_missing_audio(tmp_path):
     assert videos[0].features["audio_hit_count"] == [0.0]          # defaulted, not crashed

+def _write_windowing_only(d: str, name: str) -> None:
+    """Write a LEANER velocity-windowing/distill ``*_golden_features.json`` ? the schema the
+    R3/R4 corpus-growth path persists for clips that never went through fusion feature
+    extraction (`start`/`end` + velocity stats, NO shuttle/person/audio cue dicts). The
+    ablation loader must SKIP these, never crash on `c["shuttle"]`."""
+    cands = [{"start": 0.0, "end": 5.0, "active_frame_count": 90, "chunk_index": 0,
+              "mean_velocity": 2.0, "peak_velocity": 9.0, "track_id_count": 1}]
+    data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
+            "candidates": cands, "gts": [[0.0, 5.0]]}
+    with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as f:
+        json.dump(data, f)
+
+def test_is_fusion_schema_distinguishes_lean_and_full():
+    """The schema predicate keeps full-fusion rows (shuttle+person, audio optional) and rejects
+    the leaner velocity-windowing rows (no cue dicts); an empty candidate list is not
+    ablatable."""
+    full = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}, "audio": {}}]

```

```

+ no_audio = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}}] # audio
optional
+ lean = [{"start": 0.0, "end": 1.0, "mean_velocity": 1.0, "track_id_count": 1}]
+ assert ablation._is_fusion_schema(full) is True
+ assert ablation._is_fusion_schema(no_audio) is True
+ assert ablation._is_fusion_schema(lean) is False
+ assert ablation._is_fusion_schema([]) is False
+
+
+def test_load_videos_skips_windowing_only_schema(tmp_path):
+    """Mixed-corpus regression (the n=11 ``KeyError: 'shuttle'``): full-fusion files load; the
+    leaner velocity-windowing files (the 5 newer real clips) are SKIPPED rather than crashed
on,
+    and the litmus path runs end-to-end over the filtered ablatabl subset."""
+    d = str(tmp_path)
+    _write_learned_corpus(d, n=6) # 6 full-fusion files
(v0..v5)
+    for name in ("mahadevpura_1", "GX010128", "Badminton_BXH_2"): # 3 leaner real-clip names
+        _write_windowing_only(d, name)
+    # only the fusion-schema files load ? the leaner ones never crash the loader
+    videos = ablation.load_videos(d)
+    assert {v.name for v in videos} == {f"v{k}" for k in range(6)}
+    # ?and the litmus path (which reads the cue columns) runs cleanly on the mixed corpus
+    report = run_ablation(videos, ablation.litmus_config())
+    assert [r.signal for r in report.rows] == ["gate_a"]
+    assert report.num_videos == 6
+
+
+    # ----- the golden
litmus

    GOLDEN_DIR = os.path.join(
@@ -364,7 +406,20 @@ GOLDEN_DIR = os.path.join(

    def _golden_present() -> bool:
-    return len(glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json"))) >= 6
+    """True iff ?6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the shuttle/person
cue
+    columns). Counts only ablatabl files ? leaner velocity-windowing files are skipped by
+    ``load_videos``, so counting them would trip the litmus on an un-ablatabl corpus (the n=11
+    mixed-schema bug)."""
+    n = 0
+    for p in glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json")):
+        try:
+            with open(p, encoding="utf-8") as f:
+                cands = json.load(f).get("candidates") or []
+        except (OSError, ValueError):
+            continue
+        if ablation._is_fusion_schema(cands):
+            n += 1
+    return n >= 6

@pytest.mark.skipif(not _golden_present(),

```

4. user (2026-06-18T01:10:57.522Z)

59ae347 fix(eval): ablation litmus tolerates a mixed-schema golden corpus

5. assistant (2026-06-18T01:11:00.269Z)

Let me read the surrounding code in both files.

6. user (2026-06-18T01:11:01.911Z)

```
300         OR the gate knob, OR the structural flag. First-seen group order is preserved."""
301         order: List[str] = []
302         by_group: Dict[str, List[Signal]] = {}
303         for s in signals:
304             if s.group not in by_group:
305                 order.append(s.group)
306                 by_group[s.group] = []
307             by_group[s.group].append(s)
308         out: List[Signal] = []
309         for g in order:
310             members = by_group[g]
311             cols: Tuple[str, ...] = tuple(dict.fromkeys(c for m in members for c in
m.columns))
312             knob = next((m.gate_knob for m in members if m.gate_knob), None)
313             structural = any(m.structural for m in members)
314             out.append(Signal(name=g, group=g, columns=cols, gate_knob=knob,
structural=structural))
315         return out
316
317
318     def run_ablation(videos: Sequence[VideoCandidates], cfg: AblationConfig) ->
AblationReport:
319         """Loop the inventory at the configured granularity ? an ``AblationReport`` (sorted
table).
320
321         A pure driver: each row is one ``experiment.compare(without_s, full)``.
``granularity="group"``
322         (LOGO) ablates a whole producer's columns at once; ``"signal"`` (LOSO) one column-
set at a
323         time. The full reference is scored once implicitly per row (cheap re-scoring;
detection is
324         already persisted)."""
325         if len(videos) < 1:
326             raise experiment.ExperimentError("run_ablation needs at least one labeled
video")
327         if cfg.granularity not in ("signal", "group"):
328             raise ValueError(f"granularity must be 'signal' or 'group', got
{cfg.granularity!r}")
329
330         inventory = cfg.signals if cfg.granularity == "signal" else
_collapse_to_groups(cfg.signals)
331         rows = [ablate_one(videos, cfg, s) for s in inventory]
332
333         # Provenance: a full-set evaluation gives the honest-LOVO flag + the label-set
fingerprint.
334         full_eval = experiment.evaluate_variant(videos, cfg.full, iou=cfg.iou,
honest=cfg.honest)
335         from backend.eval.regression import gt_hash
336         all_gts = [iv for vc in videos for iv in (vc.gts or [])]
337         return AblationReport(
338             granularity=cfg.granularity,
339             iou=cfg.iou,
340             num_videos=len(videos),
341             honest_lovo=bool(full_eval["honest_lovo"]),
342             full_set_signals=[s.group for s in _collapse_to_groups(cfg.signals)],
343             full_spec_hash=cfg.full.spec_hash(),
344             label_set_hash=gt_hash(all_gts),
345             rows=rows,
346         )
347
348
349     # ----- default
inventories
```

```

350
351
352     def default_signals() -> List[Signal]:
353         """The current 4-group inventory (LOSO granularity): shuttle (velocity block +
duration),
354         gate_a (the structural net-crossings gate AND its column), person, audio.
355
356         Gate-A is marked ``structural`` (the held-shuttle FP guard) so it is reported but
never
357         flagged dead weight on "no measured lift". It ablates via the ``min_crossings`` knob
AND by
358         dropping the ``net_crossings`` column, so the leave-one-out genuinely removes the
whole cue."""
359         return [
360             Signal("shuttle.duration", "shuttle", ("duration",)),
361             Signal("shuttle.velocity", "shuttle", ("peak_velocity", "mean_velocity",
"active_ratio")),
362             Signal("gate_a", "gate_a", ("net_crossings",), gate_knob="min_crossings",
structural=True),
363             Signal("person", "person", PERSON_COLUMNS),
364             Signal("audio", "audio", AUDIO_COLUMNS),
365         ]
366
367
368     def static_gate_a_signal() -> Signal:
369         """The Gate-A-only structural signal (a pure gate knob over the recall-oriented
proposal
370         set). Ablating it against the static ``Variant(min_crossings=2)`` full reproduces
the
371         hand-measured Gate-A result (?F1 ? +0.074, 6/6, p ? 0.031) ? the framework's litmus
test."""
372         return Signal("gate_a", "gate_a", (), gate_knob="min_crossings", structural=True)
373
374
375     def litmus_config() -> AblationConfig:
376         """The Gate-A reproduction config: full = STATIC ``Variant(min_crossings=2)`` (no
learned
377         classifier), inventory = the single Gate-A gate signal. Ablating it = CONTROL+gate
vs CONTROL
378         ? exactly the historical measurement (``scratch/gate_ab_ablation.py``)."""
379         full = experiment.Variant(name="control_gate_a", min_crossings=2)
380         return AblationConfig(full=full, signals=[static_gate_a_signal()],
granularity="signal")
381
382
383     # ----- corpus
I/O
384
385
386     def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
387         """True iff every candidate row carries the per-cue feature dicts the ablation
scorer reads
388         (``shuttle`` + ``person``; ``audio`` stays optional ? older JSONs predate it).
389
390         The velocity-windowing / distill path persists a LEANER ``*_golden_features.json``
row
391         (``start``/``end`` + velocity stats, no cue dicts) for clips that never went through
fusion
392         feature extraction. ``load_videos`` SKIPS those rather than crash on
``c["shuttle"]`` ? so a
393         mixed corpus (some fusion files, some windowing-only) loads the ablatable subset
cleanly.
394         An empty candidate list is not ablatable either."""
395         return bool(candidates) and all("shuttle" in c and "person" in c for c in
candidates)

```

```

396
397
398     def load_videos(features_dir: str) -> List[VideoCandidates]:
399         """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a
400         ``VideoCandidates``
401         (intervals + shuttle + person + audio columns + golden gts). Audio columns are
402         optional
403         (older JSONs predate the audio augment) ? a missing block defaults those columns to
404         0.0 so
405         the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.
406         Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the
407         schema the
408         R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus
409         loads the
410         ablatable subset cleanly (see ``_is_fusion_schema``)."""
411         videos: List[VideoCandidates] = []
412         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
413             with open(path, encoding="utf-8") as f:
414                 d = json.load(f)
415                 cands = d["candidates"]
416                 if not _is_fusion_schema(cands):
417                     logger.info("ablation.load_videos: skipping %s ? windowing-only schema "
418                                "(no shuttle/person cue dicts), not an ablatable fusion-feature
419                                file",
420                                os.path.basename(path))
421                     continue
422                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
423                 features: Dict[str, List[float]] = {}
424                 for fn in SHUTTLE_COLUMNS:
425                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
426                 for fn in PERSON_COLUMNS:
427                     features[fn] = [float(c["person"][fn]) for c in cands]
428                 for fn in AUDIO_COLUMNS:
429                     features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
430                 gts = [(float(a), float(b)) for a, b in d["gts"]]
431                 videos.append(VideoCandidates(name=d["name"], intervals=intervals,
432                                               features=features, gts=gts))
433         return videos
434
435
436     def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
437         """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3)
438         weighted
439         by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the
440         learned path
441         (distinct from the static Gate-A litmus)."""
442         cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
443         return experiment.Variant(name="full_learned", feature_names=cols,
444                                   threshold=threshold, min_crossings=min_crossings)
445
446
447     # ----- table
448     rendering
449
450     def format_table(report: AblationReport) -> str:
451         """Sorted ASCII/markdown ablation table (by marginal ?F1 desc ? the contribution
452         ranking)."""
453         rows = report.sorted_rows()
454         header = (f"Ablation report ? full set = {{{', '.join(report.full_set_signals)}}}"
455                  f" . n={report.num_videos} . IoU {report.iou}"
456                  f" . {'LOVO-honest' if report.honest_lovo else 'static'}"
457                  f" . {report.granularity.upper()}")
458         cols = ["signal", "?F1(full?ablated)", "95% CI", "sign test", "CI?0", "verdict"]

```



```

329                                     granularity="group"))
330     out = str(tmp_path / "ablation.pdf")
331     ablation.export_pdf(report, out)
332     assert os.path.isfile(out)
333     assert os.path.getsize(out) > 1000                # non-trivial
334     reader = pypdf.PdfReader(out)
335     assert len(reader.pages) > 0
336     text = "".join(p.extract_text() for p in reader.pages)
337     assert "Ablation" in text
338     assert "clears-bar" in text                        # the verdict key rendered
339
340
341     # ----- corpus loader
342     (no GPU)
343
344     def test_load_videos_tolerates_missing_audio(tmp_path):
345         """An older feature JSON without an ``audio`` block still loads ? audio columns
346         default to
347         0.0 (loader never crashes on a pre-audio JSON)."""
348         cand = {"start": 0.0, "end": 10.0,
349                 "shuttle": {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS},
350                 "person": {fn: 0.0 for fn in ablation.PERSON_COLUMNS},
351                 "label": 1}
352         data = {"name": "noaudio", "fps": 30.0, "frame_width": 1920.0,
353                 "candidates": [cand], "gts": [[0.0, 10.0]]}
354         with open(tmp_path / "noaudio_golden_features.json", "w", encoding="utf-8") as f:
355             json.dump(data, f)
356         videos = ablation.load_videos(str(tmp_path))
357         assert len(videos) == 1
358         assert videos[0].features["audio_hit_count"] == [0.0]        # defaulted, not
359         crashed
360
361     def _write_windowing_only(d: str, name: str) -> None:
362         """Write a LEANER velocity-windowing/distill ``*_golden_features.json`` ? the schema
363         the
364         R3/R4 corpus-growth path persists for clips that never went through fusion feature
365         extraction (``start``/``end`` + velocity stats, NO shuttle/person/audio cue dicts).
366         The
367         ablation loader must SKIP these, never crash on ``c["shuttle"]``."""
368         cands = [{"start": 0.0, "end": 5.0, "active_frame_count": 90, "chunk_index": 0,
369                  "mean_velocity": 2.0, "peak_velocity": 9.0, "track_id_count": 1}]
370         data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
371                 "candidates": cands, "gts": [[0.0, 5.0]]}
372         with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as f:
373             json.dump(data, f)
374
375     def test_is_fusion_schema_distinguishes_lean_and_full():
376         """The schema predicate keeps full-fusion rows (shuttle+person, audio optional) and
377         rejects
378         the leaner velocity-windowing rows (no cue dicts); an empty candidate list is not
379         ablatable."""
380         full = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}, "audio": {}}]
381         no_audio = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}}]        #
382         audio optional
383         lean = [{"start": 0.0, "end": 1.0, "mean_velocity": 1.0, "track_id_count": 1}]
384         assert ablation._is_fusion_schema(full) is True
385         assert ablation._is_fusion_schema(no_audio) is True
386         assert ablation._is_fusion_schema(lean) is False
387         assert ablation._is_fusion_schema([]) is False

```



```

385     def test_load_videos_skips_windowing_only_schema(tmp_path):
386         """Mixed-corpus regression (the n=11 ``KeyError: 'shuttle'``): full-fusion files
load; the
387         leaner velocity-windowing files (the 5 newer real clips) are SKIPPED rather than
crashed on,
388         and the litmus path runs end-to-end over the filtered ablatable subset."""
389         d = str(tmp_path)
390         _write_learned_corpus(d, n=6)                                # 6 full-fusion files
(v0..v5)
391         for name in ("mahadevpura_1", "GX010128", "Badminton_BXH_2"): # 3 leaner real-clip
names
392             _write_windowing_only(d, name)
393             # only the fusion-schema files load ? the leaner ones never crash the loader
394             videos = ablation.load_videos(d)
395             assert {v.name for v in videos} == {f"v{k}" for k in range(6)}
396             # ?and the litmus path (which reads the cue columns) runs cleanly on the mixed
corpus
397             report = run_ablation(videos, ablation.litmus_config())
398             assert [r.signal for r in report.rows] == ["gate_a"]
399             assert report.num_videos == 6
400
401
402         # ----- the
golden litmus
403
404         GOLDEN_DIR = os.path.join(
405             os.path.dirname(os.path.abspath(__file__)), "output")
406
407
408         def _golden_present() -> bool:
409             """True iff ?6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the
shuttle/person cue
410             columns). Counts only ablatable files ? leaner velocity-windowing files are skipped
by
411             ``load_videos``, so counting them would trip the litmus on an un-ablatable corpus
(the n=11
412             mixed-schema bug)."""
413             n = 0
414             for p in glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json")):
415                 try:
416                     with open(p, encoding="utf-8") as f:
417                         cands = json.load(f).get("candidates") or []
418                     except (OSError, ValueError):
419                         continue
420                     if ablation._is_fusion_schema(cands):
421                         n += 1
422             return n >= 6
423
424
425         @pytest.mark.skipif(not _golden_present(),
426                             reason="golden output/*_golden_features.json absent (run
fusion_golden first)")
427         def test_litmus_reproduces_gate_a():
428             """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-
measured Gate-A
429             result ? marginal ?F1 ? +0.074, 6/6 wins, p ? 0.031, CI ? [+0.042, +0.104], within
tolerance.
430             This validates the whole framework against the recorded ground truth. Skips cleanly
in CI
431             where the golden substrate is absent; runs on the owner box where it is present."""
432             videos = ablation.load_videos(GOLDEN_DIR)
433             assert len(videos) == 6
434             cfg = ablation.litmus_config()
435             report = run_ablation(videos, cfg)
436             assert len(report.rows) == 1

```

```

437     row = report.rows[0]
438     assert row.signal == "gate_a"
439     # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.
440     assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
441     assert row.sign_wins == 6 and row.sign_losses == 0
442     assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
443     assert row.ci_low == pytest.approx(0.042, abs=0.01)
444     assert row.ci_high == pytest.approx(0.104, abs=0.01)
445     assert row.ci_excludes_zero is True
446     # Gate-A is structural ? reported but never auto-promoted to earns_keep.
447     assert row.verdict == ablation.STRUCTURAL_PROTECTED
448     # C4: over-segmentation drops 1.68 ? 1.01 when Gate-A is on (full).
449     assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)
450     assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)
451
452
453     @pytest.mark.skipif(not _golden_present(),
454                         reason="golden output/*_golden_features.json absent")
455     def test_litmus_report_is_well_formed():
456         """The litmus report is a proper AblationReport (provenance + a sorted single
row)."""
457         report = run_ablation(ablation.load_videos(GOLDEN_DIR), ablation.litmus_config())
458         assert isinstance(report, AblationReport)
459         assert report.num_videos == 6

```

8. assistant (2026-06-18T01:11:05.091Z)

Now let me look at the helper functions referenced in the tests (``_write_learned_corpus``, ``_learned_videos``) and the imports/types used.

9. user (2026-06-18T01:11:06.768Z)

```

1     """GPU-free tests for the LOSO/LOGO ablation runner (`backend.eval.ablation`).
2
3     Covers, all on deterministic synthetic candidate rows (no GPU, no real video):
4
5     - LOSO / LOGO correctness ? a cleanly-separating signal *earns its keep*, a noise
signal is
6         *inert*; LOGO collapses a group's columns; the ablated variant subtracts columns /
zeroes
7         the gate knob without mutating ``full``.
8     - the structural-guard exemption ? the SAME signal marked ``structural=True`` is
reported
9         (honest ?) but verdict is always ``structural_protected``, never auto-demoted on "no
lift".
10    - the honest-stats contract ? every row ships a CI + sign test, and no verdict is
stronger
11        than the stats support (the no-over-claim contract).
12    - a valid non-trivial PDF is produced (asserted via pypdf page count > 0).
13    - the golden litmus reproduction ? on the 6 real ``output/*_golden_features.json`` the
runner
14        reproduces the hand-measured Gate-A result (?F1 ? +0.074, 6/6 wins, p ? 0.031).
Skips
15        cleanly when the golden JSONs are absent (CI), runs when present (owner box).
16    """
17    from __future__ import annotations
18
19    import glob
20    import json
21    import os
22
23    import pytest
24
25    from backend.eval import ablation
26    from backend.eval.ablation import (

```

```

27     AblationConfig,
28     AblationReport,
29     Signal,
30     ablate_one,
31     run_ablation,
32 )
33 from backend.eval.experiment import Variant, VideoCandidates
34
35
36 # ----- synthetic
fixtures
37
38
39 def _gate_videos(n: int = 6, *, separating: bool = True) -> list[VideoCandidates]:
40     """`n` videos with a `net_crossings` gate signal.
41
42     ``separating=True``: rallies have net_crossings=5, junk has 0 ? so a
43     ``min_crossings=2`` gate
44     perfectly removes the junk and keeping it is load-bearing. ``separating=False``:
every window
44     has net_crossings=5, so the gate is a no-op (ablating it changes nothing ?
inert)."""
45     videos: list[VideoCandidates] = []
46     for k in range(n):
47         intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
48         junk_val = 5.0 if not separating else 0.0
49         feats = {"net_crossings": [5.0, 5.0, junk_val, junk_val]}
50         gts = [(0.0, 10.0), (20.0, 30.0)]
51         videos.append(VideoCandidates(name=f"v{k}", intervals=intervals, features=feats,
gts=gts))
52     return videos
53
54
55 def _learned_videos(n: int = 6) -> list[VideoCandidates]:
56     """`n` videos where a PERSON feature (`players_in_court`) cleanly separates
rallies from
57     junk while the shuttle/audio columns are constant (non-separating). So a learned
full-set
58     ablation must show person as the load-bearing group and the others as washes ? a
deterministic
59     LOGO/LOSO correctness fixture (mirrors `test_fusion_compare`'s construction)."""
60     videos: list[VideoCandidates] = []
61     for k in range(n):
62         intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
63         feats: dict[str, list[float]] = {}
64         for fn in ablation.SHUTTLE_COLUMNS:
65             feats[fn] = [1.0, 1.0, 1.0, 1.0] # constant ? non-
separating
66         for fn in ablation.PERSON_COLUMNS:
67             feats[fn] = [0.0, 0.0, 0.0, 0.0]
68         feats["players_in_court"] = [4.0, 4.0, 0.0, 0.0] # the separating signal
69         feats["in_court_ratio"] = [1.0, 1.0, 0.0, 0.0]
70         for fn in ablation.AUDIO_COLUMNS:
71             feats[fn] = [0.0, 0.0, 0.0, 0.0]
72         gts = [(0.0, 10.0), (20.0, 30.0)]
73         videos.append(VideoCandidates(name=f"v{k}", intervals=intervals, features=feats,
gts=gts))
74     return videos
75
76
77 # ----- LOSO
correctness
78
79
80 def test_separating_gate_earns_keep():

```

```

81     """A non-structural gate that cleanly separates rallies must EARN ITS KEEP: removing
it
82     drops F1 with the ?F1 CI clearing 0 and the sign test agreeing (6W/0L)."""
83     videos = _gate_videos(6, separating=True)
84     sig = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
85     cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[sig],
86                           granularity="signal")
87     row = ablate_one(videos, cfg, sig)
88     assert row.delta_f1 > 0.0
89     assert row.ci_excludes_zero is True
90     assert row.sign_wins == 6 and row.sign_losses == 0
91     assert row.sign_p_value <= 0.05
92     assert row.verdict == ablation.EARNS_KEEP
93     assert row.structural_protected is False
94     # C4: the gate cuts over-segmentation (more junk segments without it).
95     assert row.seg_ratio_full < row.seg_ratio_ablated
96
97
98     def test_noop_gate_is_inert():
99         """A gate whose threshold removes nothing (all windows pass) is a wash ? INERT,
never a
100         stronger claim. Honest: a single wash at n=6 is a candidate, not 'proven
useless'."""
101         videos = _gate_videos(6, separating=False)
102         sig = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
103         cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[sig],
104                               granularity="signal")
105         row = ablate_one(videos, cfg, sig)
106         assert abs(row.delta_f1) < 1e-9
107         assert row.ci_excludes_zero is False
108         assert row.verdict == ablation.INERT
109
110
111     def test_ablated_variant_does_not_mutate_full():
112         """Building the ablated variant must derive a new Variant, never mutate ``full`` in
place."""
113         full = Variant(name="full", feature_names=["a", "b", "c"], min_crossings=2)
114         before = full.to_dict()
115         ablated = ablation.ablated_variant(full, ("b",), "min_crossings")
116         assert ablated.feature_names == ["a", "c"]
117         assert ablated.min_crossings == 0
118         assert full.to_dict() == before # full untouched
119         assert full.feature_names == ["a", "b", "c"]
120

```

10. user (2026-06-18T01:11:08.037Z)

```

55: def _learned_videos(n: int = 6) -> list[VideoCandidates]:
148:     videos = _learned_videos(6)
164:     videos = _learned_videos(6)
177:     videos = _learned_videos(2)
239:     videos = _learned_videos(6)
281:     videos = _learned_videos(6)
295:     videos = _learned_videos(6)
308:     videos = _learned_videos(6)
326:     videos = _learned_videos(6)
360: def _write_windowing_only(d: str, name: str) -> None:
390:     _write_learned_corpus(d, n=6) # 6 full-fusion files
(v0..v5)
487: def _write_learned_corpus(d: str, n: int = 6) -> None:
510:     _write_learned_corpus(d)
528:     _write_learned_corpus(d)
539:     _write_learned_corpus(d, n=1)

```

11. assistant (2026-06-18T01:11:15.569Z)

Now let me look at `\_write\_learned\_corpus` (line 487) and the tests that follow, plus check the imports at the top of `ablation.py` for `Sequence`, `Dict`, `Any`, `logger`.

12. user (2026-06-18T01:11:17.437Z)

```
460         assert report.full_set_signals == ["gate_a"]
461         assert report.label_set_hash
462         out = ablation.format_table(report)
463         assert "gate_a" in out
464
465
466     # ----- edge
cases
467
468
469     def test_min_players_gate_knob_zeroed():
470         """The ``min_players`` gate knob is zeroed on ablation (the Gate-B analog)."""
471         full = Variant(name="full", min_players=2, feature_names=["a"])
472         ablated = ablation.ablated_variant(full, (), "min_players")
473         assert ablated.min_players == 0
474         assert ablated.feature_names == ["a"]          # columns untouched
475
476
477     def test_ablated_label_falls_back_to_gate_knob():
478         """With no columns, the ablated variant name carries the gate-knob label, not
479         '?'."""
479         full = Variant(name="full", min_crossings=2)
480         ablated = ablation.ablated_variant(full, (), "min_crossings")
481         assert "min_crossings" in ablated.name
482
483
484     # -----
CLI
485
486
487     def _write_learned_corpus(d: str, n: int = 6) -> None:
488         for k in range(n):
489             cands = []
490             for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0),
491 ((50.0, 51.0), 0)]:
491                 shuttle = {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS}
492                 person = {fn: 0.0 for fn in ablation.PERSON_COLUMNS}
493                 person["players_in_court"] = 4.0 if pos else 0.0
494                 person["in_court_ratio"] = 1.0 if pos else 0.0
495                 audio = {fn: 0.0 for fn in ablation.AUDIO_COLUMNS}
496                 cands.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
497                             "audio": audio, "label": pos})
498                 data = {"name": f"v{k}", "fps": 30.0, "frame_width": 1920.0,
499                       "candidates": cands, "gts": [[0.0, 10.0], [20.0, 30.0]]}
500                 with open(os.path.join(d, f"v{k}_golden_features.json"), "w", encoding="utf-8")
as f:
501                     json.dump(data, f)
502
503
504     def test_cli_main_writes_pdf_and_json(tmp_path, monkeypatch, capsys):
505         """`main()` runs the LOGO ablation and writes both a PDF and the structured
JSON."""
506         pytest.importorskip("reportlab")
507         pytest.importorskip("pypdf")
508         d = str(tmp_path / "feat")
509         os.makedirs(d)
510         _write_learned_corpus(d)
511         pdf = str(tmp_path / "out.pdf")
512         js = str(tmp_path / "out.json")
513         monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d, "--granularity",
```

```

"group",
514                                     "--pdf", pdf, "--json", js])
515     ablation.main()
516     out = capsys.readouterr().out
517     assert "Ablation report" in out
518     assert os.path.isfile(pdf) and os.path.getsize(pdf) > 1000
519     assert os.path.isfile(js)
520     report = json.loads(open(js, encoding="utf-8").read())
521     assert {r["group"] for r in report["rows"]} == {"shuttle", "gate_a", "person",
"audio"}
522
523
524     def test_cli_main_litmus_flag(tmp_path, monkeypatch, capsys):
525         """`--litmus` runs the static Gate-A config (single structural row)."""
526         d = str(tmp_path / "feat")
527         os.makedirs(d)
528         _write_learned_corpus(d)
529         monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d, "--litmus"])
530         ablation.main()
531         out = capsys.readouterr().out
532         assert "gate_a" in out
533         assert "static" in out                                     # litmus full set is the static
gate variant
534
535
536     def test_cli_main_needs_two_videos(tmp_path, monkeypatch):
537         d = str(tmp_path / "feat")
538         os.makedirs(d)
539         _write_learned_corpus(d, n=1)
540         monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d])
541         with pytest.raises(SystemExit):
542             ablation.main()
543

```

13. user (2026-06-18T01:11:18.872Z)

```

46:from __future__ import annotations
48:import argparse
49:import glob
50:import json
51:import logging
52:import os
53:from dataclasses import dataclass, field
54:from typing import Any, Dict, List, Optional, Sequence, Tuple
56:from backend.eval import experiment
57:from backend.eval.experiment import Variant, VideoCandidates
59:logger = logging.getLogger(__name__)

```

14. assistant (2026-06-18T01:11:21.844Z)

Imports are all present (`Any`, `Dict`, `Sequence`, `logger`). Now let me verify the key adversarial concerns. First, is `json.JSONDecodeError` a subclass of `ValueError`? And let me run the test suite to confirm nothing breaks.

15. user (2026-06-18T01:11:24.025Z)

True

16. user (2026-06-18T01:11:46.950Z)

```

..... [100%]
29 passed in 20.37s

```

17. assistant (2026-06-18T01:11:53.675Z)

All 29 tests pass, and `JSONDecodeError` is indeed a `ValueError` subclass. Now let me probe the specific adversarial edge cases the lens calls out. The key one: a file where SOME candidates have `shuttle` but others don't (mixed within one file), and a file with `shuttle` but missing `person`.

Let me trace the logic carefully. `\_is\_fusion\_schema` uses `all("shuttle" in c and "person" in c for c in candidates)`. So:

- A file where candidate[0] has shuttle+person but candidate[1] is lean ? `all(...)` is False ? whole file skipped. Good, no crash.
- A file where every candidate has `shuttle` but NONE has `person` ? False ? skipped. Good.
- But a file where every candidate has BOTH keys but the inner dict is missing a specific column (e.g. `shuttle` dict lacks `peak\_velocity`) ? passes `\_is\_fusion\_schema`, then crashes at line 420 `c["shuttle"][fn]`.

Let me check that last case ? whether that's a regression introduced here or pre-existing.

18. user (2026-06-18T01:11:54.425Z)

```
417         intervals = [(float(c["start"]), float(c["end"])) for c in cand]
418         features: Dict[str, List[float]] = {}
419         for fn in SHUTTLE_COLUMNS:
420             features[fn] = [float(c["shuttle"][fn]) for c in cand]
421         for fn in PERSON_COLUMNS:
422             features[fn] = [float(c["person"][fn]) for c in cand]
423         for fn in AUDIO_COLUMNS:
424             features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cand]
425         gts = [(float(a), float(b)) for a, b in d["gts"]]
426         videos.append(VideoCandidates(name=d["name"], intervals=intervals,
427                                     features=features, gts=gts))
428     return videos
```

19. assistant (2026-06-18T01:12:00.012Z)

That inner-column-missing crash at line 420/422 is **pre-existing** behavior (the old code accessed `c["shuttle"][fn]` directly too, with the same `c.get("audio")` tolerance only for audio). This change does not touch that path ? it only adds a gate that *prevents* the more common lean-schema crash. So that's not a regression introduced by this PR; it's pre-existing and out of scope. I'll note it as a `none`/observation, not a finding against this change.

Now let me verify the most important claims in the CONTEXT: does the litmus still reproduce the recorded Gate-A numbers on exactly the 6 fusion files? The golden dir is `backend/output`. Let me check whether the golden substrate is actually present in this worktree (the litmus skips when absent), and whether `\_golden\_present` behaves correctly.

20. user (2026-06-18T01:12:02.208Z)

```
Exit code 2
0
---list---
```

21. user (2026-06-18T01:12:06.203Z)

```
... [100%]
3 passed, 26 deselected in 1.97s
```

22. assistant (2026-06-18T01:12:10.406Z)

The litmus tests pass but that "3 passed" is suspicious for litmus-keyed selection ? they might be skipping. The golden dir has zero files here, so the litmus tests should skip. Let me check the skip status explicitly and verify `\_golden\_present` returns False here (which it should, so litmus is unverifiable locally ? consistent with CLAUDE.md "GPU/golden absent in dev container").

23. user (2026-06-18T01:12:14.277Z)

```
GOLDEN_DIR = C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\ablation-
schema-fix\output
_golden_present() = True
```

24. user (2026-06-18T01:12:17.559Z)

```
tests/test_ablation.py::test_litmus_reproduces_gate_a PASSED [ 33%]
tests/test_ablation.py::test_litmus_report_is_well_formed PASSED [ 66%]
tests/test_ablation.py::test_cli_main_litmus_flag PASSED [100%]
===== 3 passed, 26 deselected in 2.04s =====
```

25. assistant (2026-06-18T01:12:21.754Z)

Interesting and important. `GOLDEN\_DIR` points to ``<repo>/output`` (note: `os.path.dirname(os.path.dirname(test_file))` = repo root, then `"output"`), NOT `backend/output`. And `_golden_present()` returns `**True**` here. So the golden litmus actually RAN (PASSED, not skipped) and reproduced the Gate-A numbers. Let me confirm the golden files exist in `output/` and that the litmus genuinely loaded 6 files (the `assert len(videos) == 6`).

26. user (2026-06-18T01:12:23.893Z)

```
count:
11
names:
Badminton_BXH_2_golden_features.json
GX010128_golden_features.json
GX030094_golden_features.json
adarsh_avi_singles_golden_features.json
gbaaddy_golden_features.json
kushagra_singles_golden_features.json
largetest_doubles_golden_features.json
mahadevpura_1_golden_features.json
mahadevpura_2_golden_features.json
mahadevpura_singles_golden_features.json
testlarge_short_golden_features.json
tracked?:
```

27. assistant (2026-06-18T01:12:31.135Z)

This is the actual scenario from the bug report: `**11 golden files present**` in `output/`, a mixed corpus. They are untracked (not in git ? local owner data). Let me verify how many are full-fusion vs lean, and confirm the litmus loads exactly 6 and reproduces the recorded numbers. This is the real verification of the CONTEXT's central claim.

28. user (2026-06-18T01:12:37.099Z)

```
FULL (6):
  adarsh_avi_singles_golden_features.json
  gbaaddy_golden_features.json
  kushagra_singles_golden_features.json
  largetest_doubles_golden_features.json
  mahadevpura_singles_golden_features.json
  testlarge_short_golden_features.json
LEAN (5):
  Badminton_BXH_2_golden_features.json
  GX010128_golden_features.json
  GX030094_golden_features.json
  mahadevpura_1_golden_features.json
  mahadevpura_2_golden_features.json
EMPTY (0): []
load_videos -> 6 videos: ['adarsh_avi_singles', 'gbaaddy', 'kushagra_singles',
'largetest_doubles', 'mahadevpura_singles', 'testlarge_short']
```

29. assistant (2026-06-18T01:12:40.279Z)

35. user (2026-06-18T01:13:11.425Z)

OLD-behavior first crash on: ('Badminton_BXH_2_golden_features.json', "KeyError('shuttle')")

36. assistant (2026-06-18T01:13:17.788Z)

Confirmed: old master crashes with `KeyError('shuttle')` on `Badminton\_BXH\_2`, exactly as the bug report describes, and the fix resolves it while preserving the litmus.

Now let me address the remaining adversarial lens items thoroughly:

1. **The `>=6` gate semantics / drift.** If fusion-file count drifts. Let me examine the litmus test assertion `assert len(videos) == 6` (line 433) vs `\_golden\_present` `>= 6`. If a 7th full-fusion file is added, `\_golden\_present()` returns True but `load\_videos` returns 7, so `assert len(videos) == 6` fails. Is that pre-existing or introduced?

2. **Silent skipping** ? only info log, risk of hiding a problem.

Let me verify the `>=6` vs `==6` interaction is pre-existing (not introduced by this change).

37. user (2026-06-18T01:13:19.318Z)

```
90:     assert row.sign_wins == 6 and row.sign_losses == 0
249:         assert d["n"] == 6
289:     assert back["num_videos"] == 6
356:     assert len(videos) == 1
366: def _golden_present() -> bool:
367:     return len(glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json"))) >= 6
370: @pytest.mark.skipif(not _golden_present(),
378:     assert len(videos) == 6
386:     assert row.sign_wins == 6 and row.sign_losses == 0
398: @pytest.mark.skipif(not _golden_present(),
```

38. assistant (2026-06-18T01:13:27.900Z)

The `>=6` guard with a `==6` litmus assertion is **pre-existing** (old master line 367 `>= 6`, line 378 `assert len(videos) == 6`). The change actually **improves** the drift behavior in one direction: on old master, the presence of the 5 lean files made the raw glob count = 11 ? 6 ? `\_golden\_present()` True ? litmus runs ? **crashes** in `load\_videos` (KeyError). The new `\_golden\_present()` counts only the 6 **fusion** files, so it gates correctly on ablatable count. That's a net improvement, not a fragility introduced.

The latent fragility ? if a **7th full-fusion** file is added, `\_golden\_present()` returns True but `load\_videos` returns 7 and `assert len(videos) == 6` (line 433) fails ? is **pre-existing** and unchanged by this PR. Worth a should-fix/nit note but it does not block this fix.

Now let me run the full repo test suite and lint/type checks per the pre-LGTM gate, to make sure nothing else regressed.

39. user (2026-06-18T01:13:28.900Z)

All checks passed!
=== ruff exit: 0 ===

40. user (2026-06-18T01:13:31.271Z)

```
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 1 source file
=== mypy exit: 0 ===
```

41. assistant (2026-06-18T01:13:34.460Z)

Ruff and mypy clean. Let me run the broader test suite to confirm no regression elsewhere (the change touches only ablation, but the pre-LGTM gate requires the full suite). Let me check what `run\_all\_tests.py` does and run a reasonable subset given time, plus the eval tests.

42. user (2026-06-18T01:15:03.363Z)

```
..... [ 8%]
..... [ 16%]
..... [ 25%]
..... [ 33%]
..... [ 42%]
..... [ 50%]
..... [ 59%]
..... [ 67%]
.....SSS [ 75%]
SS..... [ 84%]
..... [ 92%]
..... [100%]
848 passed, 5 skipped in 84.48s (0:01:24)
```

43. assistant (2026-06-18T01:15:06.497Z)

Full suite: 848 passed, 5 skipped, 0 failed. Let me verify the 5 skips are unrelated to this change (expected env-based skips like reportlab/pypdf/torch), so the skips aren't masking a problem introduced here.

44. user (2026-06-18T01:16:35.206Z)

```
SKIPPED [1] tests\test_served_gate_a_regression.py:74: golden trajectories absent (manifest
'C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\.claude\\worktrees\\ablation-schema-
fix\\output\\human_lovo_manifest.json'); runs on the owner box
SKIPPED [1] tests\test_served_gate_a_regression.py:85: golden trajectories absent (manifest
'C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\.claude\\worktrees\\ablation-schema-
fix\\output\\human_lovo_manifest.json'); runs on the owner box
SKIPPED [1] tests\test_served_gate_a_regression.py:102: golden trajectories absent (manifest
'C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\.claude\\worktrees\\ablation-schema-
fix\\output\\human_lovo_manifest.json'); runs on the owner box
SKIPPED [1] tests\test_served_gate_a_regression.py:114: golden trajectories absent (manifest
'C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\.claude\\worktrees\\ablation-schema-
fix\\output\\human_lovo_manifest.json'); runs on the owner box
SKIPPED [1] tests\test_served_gate_a_regression.py:132: golden trajectories absent (manifest
'C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\.claude\\worktrees\\ablation-schema-
fix\\output\\human_lovo_manifest.json'); runs on the owner box
848 passed, 5 skipped in 83.75s (0:01:23)
```

45. assistant (2026-06-18T01:16:41.693Z)

The 5 skips are all in `test\_served\_gate\_a\_regression.py` due to absent golden trajectory manifest (owner-box only) ? unrelated to this change. Notably, the ablation golden litmus tests did NOT skip ? they ran (because the 11 golden files are present locally in `output/`) and passed, which means I have actually verified the central claim end-to-end on the real golden substrate, not just a synthetic mock.

Let me do one final adversarial check on the remaining lens items: malformed/garbage JSON behavior in `load\_videos` (not just `\_golden\_present`). The `\_golden\_present` swallows `OSError`, `ValueError`, but `load\_videos` itself does NOT wrap `json.load` ? so a garbage JSON in the dir would crash the CLI. Is that a regression or pre-existing? And does `\_golden\_present`'s except correctly handle the empty-candidates / non-dict case?

46. user (2026-06-18T01:16:43.455Z)

----- corpus I/O

```

def load_videos(features_dir: str) -> List[VideoCandidates]:
    """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals + shuttle +
    person + audio columns + golden gts). Audio columns are optional (older JSONs predate the
    audio augment) ? a missing block defaults those columns to 0.0 so the loader never crashes,
    mirroring ``experiment._feature_column``'s tolerance."""
    videos: List[VideoCandidates] = []
    for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
        with open(path, encoding="utf-8") as f:
            d = json.load(f)
            cands = d["candidates"]
            intervals = [(float(c["start"]), float(c["end"])) for c in cands]
            features: Dict[str, List[float]] = {}
            for fn in SHUTTLE_COLUMNS:
                features[fn] = [float(c["shuttle"][fn]) for c in cands]
            for fn in PERSON_COLUMNS:
                features[fn] = [float(c["person"][fn]) for c in cands]
            for fn in AUDIO_COLUMNS:
                features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
            gts = [(float(a), float(b)) for a, b in d["gts"]]
            videos.append(VideoCandidates(name=d["name"], intervals=intervals,
                                         features=features, gts=gts))

    return videos

def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
    """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3) weighted
    by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the learned path
    (distinct from the static Gate-A litmus)."""
    cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
    return experiment.Variant(name="full_learned", feature_names=cols,
                             threshold=threshold, min_crossings=min_crossings)

```

----- table rendering

47. assistant (2026-06-18T01:16:53.069Z)

Confirmed: `load\_videos` never wrapped `json.load` in try/except on old master either ? a garbage JSON would crash the loader pre-existing. This change doesn't alter that path. Not a regression.

One subtle point on `\_golden\_present`'s try/except: it does `json.load(f).get("candidates")`. If the JSON is valid but the top-level is a **list** (not a dict), `.get` raises `AttributeError`, which is NOT caught by `(OSError, ValueError)` ? it would propagate. But this is a defensive test-helper for the skip gate; a top-level non-dict golden file would be a corrupt fixture, and this is test-only code (`_golden_present` is in the test file, gating a skipif). The previous version (`len(glob(...)) >= 6`) never opened files at all so couldn't hit this. This is a minor robustness nit in test code, very low severity ? the realistic failure modes (missing file, bad JSON) ARE caught, and `json.JSONDecodeError` is a `ValueError` subclass as I verified. I'll note it as a nit.

Let me also double-check: the new `test_load_videos_skips_windowing_only_schema` writes 6 + 3 = 9 files but asserts only v0..v5 load. Confirmed it passed. And `_is_fusion_schema` handles the "mixed within one file" case ? let me verify that one edge directly since the lens specifically calls it out.

48. user (2026-06-18T01:16:59.487Z)

```

mixed-within-file is_fusion -> False
shuttle-no-person is_fusion -> False

```

49. assistant (2026-06-18T01:17:09.486Z)

Both edge cases handled correctly: a file with mixed candidates (some full, some lean) ? `False`

? skipped, no crash. A file with `shuttle` but no `person` ? `False` ? skipped. The `all(...)` semantics correctly require *every* candidate to be fully-formed, which is conservative and safe (it skips a partially-corrupt file rather than half-loading it).

I have thoroughly verified the change. Summary of my adversarial findings:

- **Central claim verified end-to-end on the REAL golden substrate** (not mocks): 11 golden files in `output/` (6 full-fusion + 5 lean), `load\_videos` cleanly loads the 6, and the litmus reproduces $F1=0.0739$, $6W/0L$, $p=0.0312$, $CI=[0.042,0.104]$, seg 1.01/1.68 ? exact match to recorded Gate-A numbers.
- Old master demonstrably crashes with `KeyError('shuttle')` on `Badminton\_BXH\_2`; the fix resolves it.
- Full suite: 848 passed, 5 skipped (unrelated owner-box golden trajectory skips), ruff clean, mypy clean.
- All lens edge cases (empty list, mixed-within-file, shuttle-without-person, missing audio) handled correctly.
- The only items I found are pre-existing latent issues unchanged or improved by this PR (the `>=6`/`==6`` drift, unguarded `json.load` in `load\_videos`, and a minor non-dict `.get` gap in the test-only `\_golden\_present`).

No must-fix found. This is a clean, well-tested, narrowly-scoped fix.

50. user (2026-06-18T01:17:33.379Z)

Structured output provided successfully

51. assistant (2026-06-18T01:17:51.484Z)

I've completed the adversarial review. The fix is sound and merge-worthy.

Verdict: does not block merge. I actively tried to break it and could not find a must-fix.

Key verification (done against the REAL golden substrate, not just mocks ? the litmus tests actually ran here rather than skipping, because `output/` locally holds the exact n=11 mixed corpus from the bug report):

- 11 golden files = 6 full-fusion + 5 lean (`Badminton\_BXH\_2`, `GX010128`, `GX030094`, `mahadevpura\_1`, `mahadevpura\_2`). `load\_videos()` cleanly returns the 6 ablatable videos.
- Litmus reproduces the recorded Gate-A numbers exactly: $F1=0.0739$, $6W/0L$, $p=0.0312$, $CI=[0.042,0.104]$, seg 1.01/1.68 ? all within the test tolerances at `tests/test\_ablation.py:440-450`.
- Confirmed old `origin/master` crashes with `KeyError('shuttle')` on `Badminton\_BXH\_2`; the fix removes it.
- Full repo suite: 848 passed, 5 skipped (unrelated owner-box golden-trajectory skips), `ruff` clean, `mypy` clean.
- All lens edge cases pass: empty list ? not fusion; mixed-within-one-file ? `all()` returns False ? whole file skipped (no half-load); shuttle-but-no-person ? skipped; missing audio ? defaults to 0.0. `json.JSONDecodeError` is a `ValueError` subclass, so `\_golden\_present`'s `(OSError, ValueError)` catches the realistic failures without masking logic errors.

The only items I flagged are pre-existing and untouched (or improved) by this PR:

- **should-fix (pre-existing):** `_golden_present() >= 6`` vs litmus `assert len(videos) == 6`` ? adding a 7th *fusion* file would break the litmus. This PR actually fixes the opposite drift.
- **nit (test-only):** `_golden_present`'s` .get('candidates')`` would raise an uncaught `AttributeError` on a valid-but-non-dict top-level JSON.
- **none:** `load_videos`` still has unguarded `json.load`` and direct inner-column access ? both pre-existing on master, not regressions.

Relevant files: `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix/backend/eval/ablation.py`` (lines 386-428) and `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/ablation-schema-fix/tests/test_ablation.py`` (lines 360-422).